Status	Finished
Started	Thursday, 8 October 2026, 12:09 PM
Completed	Thursday, 8 October 2026, 12:27 PM
Duration	17 mins 30 secs
Grade	**10.00** out of 10.00 (**100**%)

Question **1**

Correct

Mark 1.00 out of 1.00

Create a student dictionary for n students with the student name as key and their test mark assignment mark and lab mark as values. Do the following computations and display the result.

1.Identify the student with the highest average score

2.Identify the student who as the highest Assignment marks

3.Identify the student with the Lowest lab marks

4.Identify the student with the lowest average score

Note:

If more than one student has the same score display all the student names

Sample input:

4

James 67 89 56

Lalith 89 45 45

Ram 89 89 89

Sita 70 70 70

Sample Output:

Ram

James Ram

Lalith

Lalith

For example:

Input	Result
4 James 67 89 56 Lalith 89 45 45 Ram 89 89 89 Sita 70 70 70	Ram James Ram Lalith Lalith

Answer: (penalty regime: 0 %)

```
import sys

def main():
    data = sys.stdin.read().splitlines()
    idx = 0
    # skip blank lines before n
    while idx < len(data) and not data[idx].split():
        idx += 1
    if idx >= len(data):
        return
    n = int(data[idx].split()[0])
    idx += 1

    students = {}
    count = 0
    while count < n and idx < len(data):
        parts = data[idx].split()
        idx += 1
        if len(parts) < 4:            # skip blank/malformed lines
```

```
            continue
        name = ' '.join(parts[:-3])   # supports multi-word names too
        test, assign, lab = map(float, parts[-3:])
        students[name] = (test, assign, lab)
        count += 1

    if not students:
        return

    total = {name: m[0] + m[1] + m[2] for name, m in students.items()}

    def show(names):
        print(' '.join(sorted(names)))    # ties -> alphabetical order

    # 1. highest average (comparing totals == comparing averages)
    max_total = max(total.values())
    show([s for s in total if total[s] == max_total])

    # 2. highest assignment mark
    max_assign = max(m[1] for m in students.values())
    show([s for s, m in students.items() if m[1] == max_assign])

    # 3. lowest lab mark
    min_lab = min(m[2] for m in students.values())
    show([s for s, m in students.items() if m[2] == min_lab])

    # 4. lowest average
    min_total = min(total.values())
    show([s for s in total if total[s] == min_total])

main()
```

	Input	Expected	Got	
✓	4 James 67 89 56 Lalith 89 45 45 Ram 89 89 89 Sita 70 70 70	Ram James Ram Lalith Lalith	Ram James Ram Lalith Lalith	✓
✓	3 Raja 95 67 90 Aarav 89 90 90 ShadhanA 95 95 91	ShadhanA ShadhanA Aarav Raja Raja	ShadhanA ShadhanA Aarav Raja Raja	✓

Passed all tests! ✓

Correct

Marks for this submission: 1.00/1.00.

Question **2**

Correct

Mark 1.00 out of 1.00

You are given a string `word`. A letter is called **special** if it appears both in lowercase and uppercase in `word`.

Your task is to return the number of **special** letters in `word`.

Constraints

- The input string `word` will contain only alphabetic characters (both lowercase and uppercase).
- The solution must utilize a dictionary to determine the number of special letters.
- The function should handle various edge cases, such as strings without any special letters, strings with only lowercase or uppercase letters, and mixed strings.

Examples

Example 1:

Input: word = "aaAbcBC"
Output: 3

Explanation:
The special characters in `word` are 'a', 'b', and 'c'.

Example 2:

Input: word = "abc"
Output: 0

Explanation:
No character in `word` appears in uppercase.

For example:

Test	Result
`print(count_special_letters("AaBbCcDdEe"))`	`5`

Answer: (penalty regime: 0 %)

Reset answer

```
def count_special_letters(word: str) -> int:
    # Dictionary to track character casing status
    # Key: lowercase char -> Value: set of cases seen, e.g., {'l', 'u'}
    char_cases = {}

    for char in word:
        lower_char = char.lower()
        if lower_char not in char_cases:
            char_cases[lower_char] = set()

        if char.isupper():
            char_cases[lower_char].add('u')
        else:
            char_cases[lower_char].add('l')

    # Count keys that have both lowercase ('l') and uppercase ('u') recorded
    special_count = 0
    for cases in char_cases.values():
        if 'l' in cases and 'u' in cases:
            special_count += 1

    return special_count

```

	Test	Expected	Got	
✓	`print(count_special_letters("AaBbCcDdEe"))`	`5`	`5`	✓
✓	`print(count_special_letters("ABCDE"))`	`0`	`0`	✓

	Test	Expected	Got	
✓	`print(count_special_letters("abcde"))`	`0`	`0`	✓

Passed all tests! ✓

Correct

Marks for this submission: 1.00/1.00.

Question **3**

Correct

Mark 1.00 out of 1.00

Given a number, convert it into corresponding alphabet.

Input	Output
1	A
26	Z
27	AA
676	YZ

Input Format

Input is an integer

Output Format

Print the alphabets

Constraints

1 <= num <= 4294967295

Sample Input 1

26

Sample Output 1

Z

For example:

Test	Result
`print(excelNumber(26))`	Z

Answer: (penalty regime: 0 %)

Reset answer

```
def excelNumber(num: int) -> str:
    result = []

    while num > 0:
        # Subtract 1 to adjust for 1-based indexing (A=1, B=2...)
        num -= 1

        # Get the remainder to determine the current character (0 = A, 25 = Z)
        remainder = num % 26
        result.append(chr(65 + remainder))

        # Move to the next positional digit
        num //= 26

    # Reverse the array since we built the string from right to left
    return "".join(reversed(result))

```

	Test	Expected	Got	
✓	`print(excelNumber(26))`	Z	Z	✓

	Test	Expected	Got	
✓	`print(excelNumber(27))`	AA	AA	✓

Passed all tests! ✓

Correct

Marks for this submission: 1.00/1.00.

Question **4**

Correct

Mark 1.00 out of 1.00

In the game of Scrabble™, each letter has points associated with it. The total score of a word is the sum of the scores of its letters. More common letters are worth fewer points while less common letters are worth more points. The points associated with each letter are shown below:

Points Letters

1 A, E, I, L, N, O, R, S, T and U

2 D and G

3 B, C, M and P

4 F, H, V, W and Y

5 K

8 J and X

10 Q and Z

Write a program that computes and displays the Scrabble™ score for a word. Create a dictionary that maps from letters to point values. Then use the dictionary to compute the score.

A Scrabble™ board includes some squares that multiply the value of a letter or the value of an entire word. We will ignore these squares in this exercise.

Sample Input

REC

Sample Output

REC is worth 5 points.

For example:

Input	Result
REC	REC is worth 5 points.

Answer: (penalty regime: 0 %)

```
import sys

def calculate_scrabble_score():
    # Define the point mappings as specified
    points_map = {
        'A': 1, 'E': 1, 'I': 1, 'L': 1, 'N': 1, 'O': 1, 'R': 1, 'S': 1, 'T': 1, 'U': 1,
        'D': 2, 'G': 2,
        'B': 3, 'C': 3, 'M': 3, 'P': 3,
        'F': 4, 'H': 4, 'V': 4, 'W': 4, 'Y': 4,
        'K': 5,
        'J': 8, 'X': 8,
        'Q': 10, 'Z': 10
    }

    # Read the input word, strip whitespace, and handle potential empty inputs
    word = sys.stdin.read().strip()
    if not word:
        return

    # Convert word to uppercase to safely match dictionary keys
    score = sum(points_map.get(letter, 0) for letter in word.upper())

    # Display the result exactly matching the sample output format
    print(f"{word} is worth {score} points.")

if __name__ == '__main__':
    calculate_scrabble_score()

```

	Input	Expected	Got	
✓	GOD	GOD is worth 5 points.	GOD is worth 5 points.	✓
✓	REC	REC is worth 5 points.	REC is worth 5 points.	✓

Passed all tests! ✓

Correct

Marks for this submission: 1.00/1.00.

Question **5**

Correct

Mark 1.00 out of 1.00

Given an array of names of candidates in an election. A candidate name in the array represents a vote cast to the candidate. Print the name of candidates received Max vote. If there is tie, print a lexicographically smaller name.

Examples:

Input : votes[] = {"john", "johnny", "jackie",

"johnny", "john", "jackie",

"jamie", "jamie", "john",

"johnny", "jamie", "johnny",

"john"};

Output : John

We have four Candidates with name as 'John', 'Johnny', 'jamie', 'jackie'. The candidates John and Johny get maximum votes. Since John is alphabetically smaller, we print it. Use dictionary to solve the above problem

Sample Input:

10

John

John

Johny

Jamie

Jamie

Johny

Jack

Johny

Johny

Jackie

Sample Output:

Johny

Answer: (penalty regime: 0 %)

```
import sys

def find_election_winner():
    # Read all inputs from standard input to handle hidden whitespaces cleanly
    input_data = sys.stdin.read().splitlines()
    if not input_data:
        return

    # First line contains the number of votes
```

```
    n = int(input_data[0].strip())

    # Dictionary to keep track of vote counts for each candidate
    vote_counts = {}

    # Read votes based on the count n
    for i in range(1, n + 1):
        if i < len(input_data):
            candidate = input_data[i].strip()
            if not candidate:
                continue

            # Count the votes using a dictionary
            if candidate in vote_counts:
                vote_counts[candidate] += 1
            else:
                vote_counts[candidate] = 1

    # Initialize tracking variables for the winner
    max_votes = -1
    winner = ""

    # Find the candidate with maximum votes and apply lexicographical tie-breaking
    for candidate, votes in vote_counts.items():
        if votes > max_votes:
            max_votes = votes
            winner = candidate
        elif votes == max_votes:
            # If there's a tie, pick the lexicographically (alphabetically) smaller name
            if candidate < winner:
                winner = candidate

    # Print the winning candidate
    print(winner)

if __name__ == '__main__':
    find_election_winner()

```

	Input	Expected	Got	
✓	10 John John Johny Jamie Jamie Johny Jack Johny Johny Jackie	Johny	Johny	✓
✓	6 Ida Ida Ida Kiruba Kiruba Kiruba	Ida	Ida	✓

Passed all tests! ✓

Correct

Marks for this submission: 1.00/1.00.

Question **6**

Correct

Mark 1.00 out of 1.00

Give a dictionary with value lists, sort the keys by summation of values in value list.

Input : test_dict = {'Gfg' : [6, 7, 4], 'best' : [7, 6, 5]}

Output : {'Gfg': 17, 'best': 18}

Explanation : Sorted by sum, and replaced.

Input : test_dict = {'Gfg' : [8,8], 'best' : [5,5]}

Output : {'best': 10, 'Gfg': 16}

Explanation : Sorted by sum, and replaced.

Sample Input:

2

Gfg 6 7 4

Best 7 6 5

Sample Output

Gfg 17

Best 18

For example:

Input	Result
2 Gfg 6 7 4 Best 7 6 5	Gfg 17 Best 18

Answer: (penalty regime: 0 %)

```
import sys

def sort_dict_by_sum():
    # Read all lines from standard input
    input_data = sys.stdin.read().splitlines()
    if not input_data:
        return

    # First line contains the number of items
    n = int(input_data[0].strip())

    # Process each line and build a dictionary mapping keys to their sum
    sum_dict = {}
    for i in range(1, n + 1):
        if i < len(input_data):
            line = input_data[i].strip()
            if not line:
                continue
            parts = line.split()
            key = parts[0]
            # Convert the remaining items in the line to integers and sum them
            values_sum = sum(int(x) for x in parts[1:])
            sum_dict[key] = values_sum

    # Sort the items by their sum value (x[1])
    sorted_items = sorted(sum_dict.items(), key=lambda x: x[1])

    # Display the result exactly matching the sample output format
    for key, total in sorted_items:
        print(f"{key} {total}")

if __name__ == '__main__':
    sort_dict_by_sum()

```

	Input	Expected	Got	
✓	2 Gfg 6 7 4 Best 7 6 5	Gfg 17 Best 18	Gfg 17 Best 18	✓
✓	2 Gfg 6 6 Best 5 5	Best 10 Gfg 12	Best 10 Gfg 12	✓

Passed all tests! ✓

Correct

Marks for this submission: 1.00/1.00.

Question **7**

Correct

Mark 1.00 out of 1.00

Objective:

Develop a Python program that takes an input string from the user and counts the number of occurrences of each vowel (a, e, i, o, u) in the string. The program should be case-insensitive, meaning it should treat uppercase and lowercase vowels as the same.

Description:

Vowels play a significant role in the English language and other alphabet-based languages. Counting vowels in a given string is a fundamental task that can be applied in various text processing applications, including speech recognition, linguistic research, and text analysis. The objective of this problem is to create a Python script that accurately counts and displays the number of times each vowel appears in a user-provided string.

Program Requirements:

Input:

First line reading String as input, The string can contain any characters, including letters, numbers, and special characters.

Output:

Display the number of occurrences of each vowel in the string.

The output should list each vowel followed by its count.

Example:

Consider the following example for better understanding:

- **Input:** "Python Programming"
- **Output**

  a = 1
  e = 0
  i = 1
  o = 2
  u = 0

For example:

Input	Result
Hello World	a = 0 e = 1 i = 0 o = 2 u = 0
Python	a = 0 e = 0 i = 0 o = 1 u = 0

Answer: (penalty regime: 0 %)

```
import sys

def count_vowels():
    # Read the entire input line from standard input
    input_text = sys.stdin.read()
    if not input_text:
        return

    # Convert text to lowercase to ensure case-insensitivity
    text_lower = input_text.lower()

    # Initialize the ordered list of vowels to track
    vowels = ['a', 'e', 'i', 'o', 'u']

    # Count occurrences using the string count method for each vowel
    for vowel in vowels:
        count = text_lower.count(vowel)
        print(f"{vowel} = {count}")

```

```
20 if __name__ == '__main__':
21     count_vowels()
22
```

	Input	Expected	Got	
✓	Hello World	a = 0 e = 1 i = 0 o = 2 u = 0	a = 0 e = 1 i = 0 o = 2 u = 0	✓
✓	AEIOU aeio u	a = 2 e = 2 i = 2 o = 2 u = 2	a = 2 e = 2 i = 2 o = 2 u = 2	✓
✓	Python	a = 0 e = 0 i = 0 o = 1 u = 0	a = 0 e = 0 i = 0 o = 1 u = 0	✓
✓	abcdefghijklmnopqrstuvwxyz	a = 1 e = 1 i = 1 o = 1 u = 1	a = 1 e = 1 i = 1 o = 1 u = 1	✓
✓	12345!@#$%AEIOU	a = 1 e = 1 i = 1 o = 1 u = 1	a = 1 e = 1 i = 1 o = 1 u = 1	✓

Passed all tests! ✓

Correct

Marks for this submission: 1.00/1.00.

Question **8**

Correct

Mark 1.00 out of 1.00

A sentence is a string of single-space separated words where each word consists only of lowercase letters.A word is uncommon if it appears exactly once in one of the sentences, and does not appear in the other sentence.

Given two sentences s1 and s2, return a list of all the uncommon words. You may return the answer in any order.

Example 1:

Input: s1 = "this apple is sweet", s2 = "this apple is sour"

Output: ["sweet","sour"]

Example 2:

Input: s1 = "apple apple", s2 = "banana"

Output: ["banana"]

Constraints:

1 <= s1.length, s2.length <= 200

s1 and s2 consist of lowercase English letters and spaces.

s1 and s2 do not have leading or trailing spaces.

All the words in s1 and s2 are separated by a single space.

Note:

Use dictionary to solve the problem

For example:

Input	Result
this apple is sweet this apple is sour	sweet sour

Answer: (penalty regime: 0 %)

```
import sys

def find_uncommon_words():
    # Read all lines from standard input and filter out any completely blank lines
    input_data = sys.stdin.read().splitlines()
    clean_lines = [line.strip() for line in input_data if line.strip()]

    if len(clean_lines) < 2:
        return

    s1 = clean_lines[0]
    s2 = clean_lines[1]

    word_counts = {}
    order_of_appearance = []

    # Process both sentences sequentially to capture exact word order
    for sentence in [s1, s2]:
        for word in sentence.split():
            if word not in word_counts:
                order_of_appearance.append(word)
            word_counts[word] = word_counts.get(word, 0) + 1

    # Filter using our order_of_appearance list to keep original sentence sequence
    uncommon_words = [word for word in order_of_appearance if word_counts[word] == 1]

    # Print the result separated by a single space
    print(" ".join(uncommon_words))

if __name__ == '__main__':
    find_uncommon_words()

```

	Input	Expected	Got	
✓	this apple is sweet this apple is sour	sweet sour	sweet sour	✓
✓	apple apple banana	banana	banana	✓

Passed all tests! ✓

Correct

Marks for this submission: 1.00/1.00.

Question **9**

Correct

Mark 1.00 out of 1.00

A company wants to send its quotation secretly to its client. The company decided to encrypt the amount they are sending to their client with some special symbols so that the equation amount will not be revealed to any external person. They used the special symbols !,@,#,$,%,^,&,*,>,< for 0,1,2,3,4,5,6,7,8,9 respectively. Write a python code to help the company to convert the amount to special symbols.

(Value rounded off to 2 decimal points)

Input

n: Float data type which reads amount to send

Output

s: : String data type which displays symbols

Sample Testcase 1

Input

10000

Output

@!!!!.!!

Sample Testcase2

1234.56

Output

@#$%.^&

For example:

Input	Result
1345.23	@$%^.#$
15000.59	@^!!!.^<
156789	@^&*><.!!

Answer: (penalty regime: 0 %)

```
import sys

def encrypt_amount():
    # Read all input data from standard input
    input_data = sys.stdin.read().strip()
    if not input_data:
        return

    # Convert input to a float data type
    amount = float(input_data)

    # Round the value off to exactly 2 decimal points
    # This guarantees the format has a dot and two decimal places (e.g., 10000 -> "10000.00")
    formatted_amount = f"{amount:.2f}"

    # Define the mapping dictionary according to the problem statement
    symbol_map = {
        '0': '!',
        '1': '@',
        '2': '#',
        '3': '$',
        '4': '%',
        '5': '^',
        '6': '&',
        '7': '*',
        '8': '>',
        '9': '<',
        '.': '.'  # The decimal point remains unchanged
    }

    # Convert each character of the formatted string to its corresponding special symbol
    encrypted_chars = [symbol_map[char] for char in formatted_amount]
```

```
    encrypted_string = "".join(encrypted_chars)

    # Print the resulting encrypted string
    print(encrypted_string)

if __name__ == '__main__':
    encrypt_amount()
```

	Input	Expected	Got	
✓	1345.23	@$%^.#$	@$%^.#$	✓
✓	15000.59	@^!!!.^<	@^!!!.^<	✓
✓	1234	@#$%.!!	@#$%.!!	✓
✓	156789	@^&*><.!!	@^&*><.!!	✓

Passed all tests! ✓

Correct

Marks for this submission: 1.00/1.00.

Question **10**

Correct

Mark 1.00 out of 1.00

A sentence is a list of words that are separated by a single space with no leading or trailing spaces. Each word consists of lowercase and uppercase English letters.

A sentence can be shuffled by appending the 1-indexed word position to each word then rearranging the words in the sentence.

For example, the sentence "This is a sentence" can be shuffled as "sentence4 a3 is2 This1" or "is2 sentence4 This1 a3".

Given a shuffled sentence s containing no more than 9 words, reconstruct and return the original sentence.

Example 1:

Input:

is2 sentence4 This1 a3

Output:

This is a sentence

Explanation: Sort the words in s to their original positions "This1 is2 a3 sentence4", then remove the numbers.

Example 2:

Input:

Myself2 Me1 I4 and3

Output:

Me Myself and I

Explanation: Sort the words in s to their original positions "Me1 Myself2 and3 I4", then remove the numbers.

Constraints:

2 <= s.length <= 200

s consists of lowercase and uppercase English letters, spaces, and digits from 1 to 9.

The number of words in s is between 1 and 9.

The words in s are separated by a single space.

s contains no leading or trailing spaces.

Answer: (penalty regime: 0 %)

```
import sys

def reconstruct_sentence():
    # Read the shuffled sentence from standard input
    input_data = sys.stdin.read().strip()
    if not input_data:
        return

    # Split the sentence into individual shuffled words
    shuffled_words = input_data.split()

    # Dictionary to map the original 1-indexed position to the cleaned word
    # Key: position (int) -> Value: word (str)
    ordered_words = {}

    for item in shuffled_words:
        # Extract the position digit from the end of the word string
        position = int(item[-1])
        # Strip the trailing position digit to get the clean word
        clean_word = item[:-1]

        ordered_words[position] = clean_word

    # Reconstruct the sentence by pulling words from position 1 up to the total count
    result_words = [ordered_words[i] for i in range(1, len(shuffled_words) + 1)]

    # Print the final reconstructed sentence joined by single spaces
```

```
    print(" ".join(result_words))

if __name__ == '__main__':
    reconstruct_sentence()
```

	Input	Expected	Got	
✓	is2 sentence4 This1 a3	This is a sentence	This is a sentence	✓

Passed all tests! ✓

Correct

Marks for this submission: 1.00/1.00.